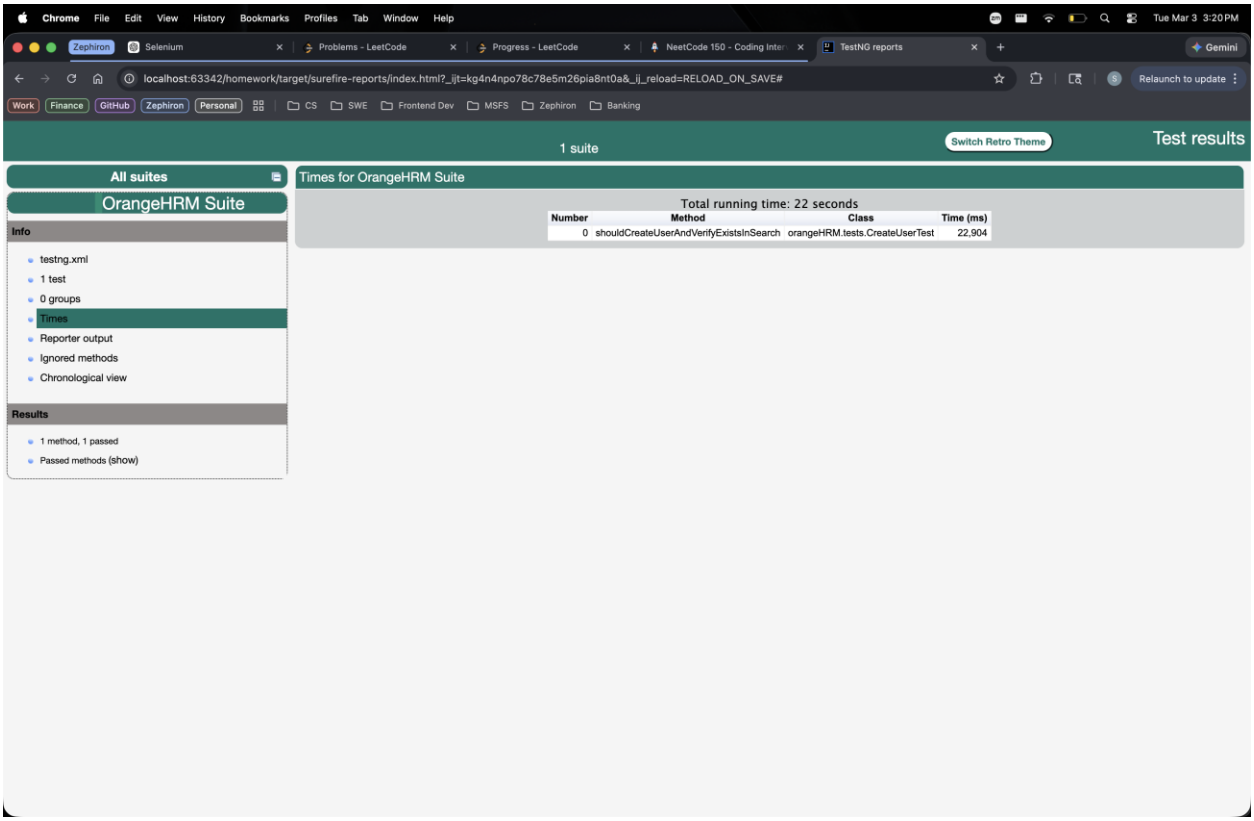
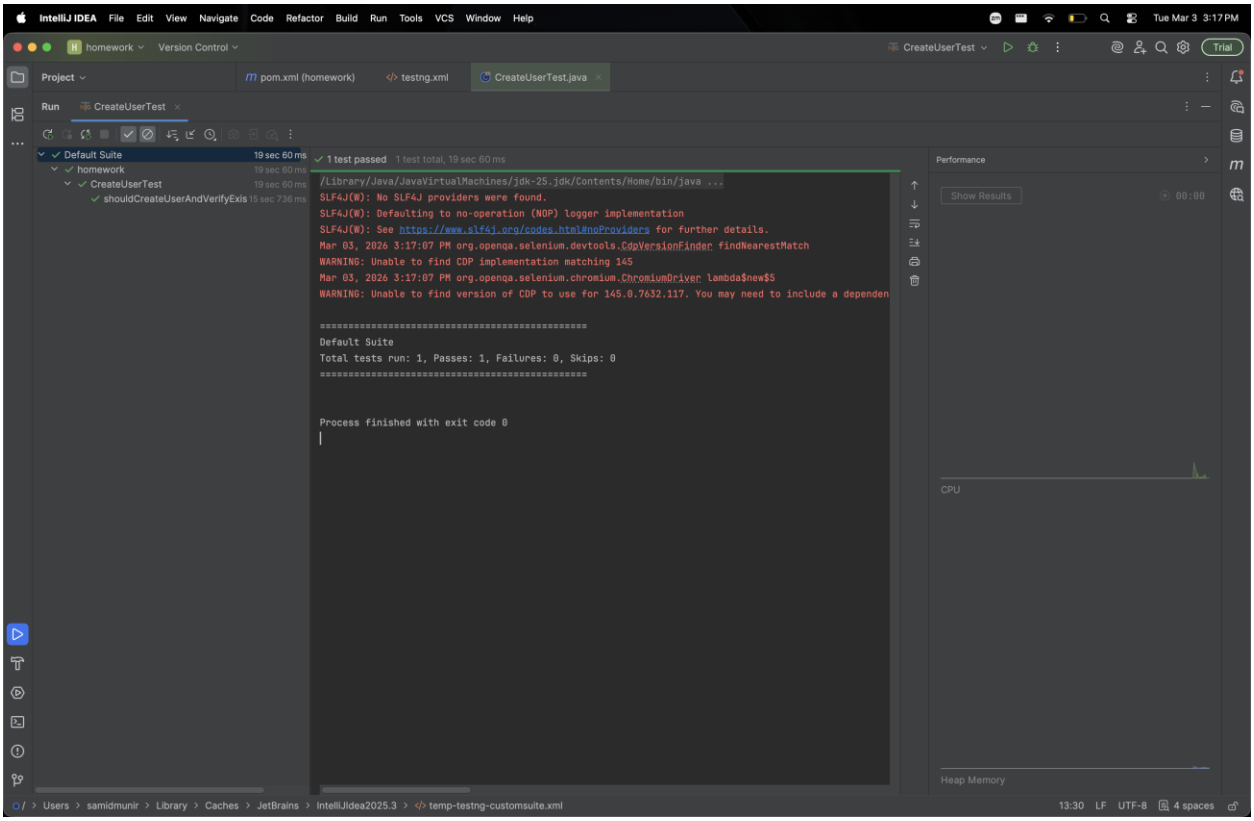
Week 3 – Day 1 Assignment (February 27th, 2026)





```java
public class CreateUserTest {
    private void waitForLoaderToDisappear() {
            // not fatal
        }
    }

    private void selectFromCustomDropdown(By dropdownDisplay, String optionText) {
        click(dropdownDisplay);
        By option = By.xpath( xpathExpression: "//div[@role='listbox']//span[normalize-space()='" + optionText + "']");
        click(option);
    }

    private void selectEmployeeFromAutocomplete(String partial) {
        clearAndType(employeeNameInput, partial);

        By firstOption = By.xpath( xpathExpression: "//div[@role='listbox']//span[1]");
        waitVisible(firstOption);
        click(firstOption);

        String value = driver.findElement(employeeNameInput).getAttribute( name: "value");
        Assert.assertTrue( condition: value != null && !value.trim().isEmpty(),
                message: "Employee Name was not selected properly from autocomplete.");
    }

    private By rowByUsername(String username) {
        return By.xpath( xpathExpression: "//div[@class='oxd-table-body']//div[@role='row']" +
                "[.//div[@role='cell'][2][normalize-space()='" + username + "']]");
    }

    private boolean isMac() {
        String os = System.getProperty( key: "os.name",  def: "");
        return os.toLowerCase().contains("mac");
    }

    private void slowMo() {
        if (!SLOW_MO) return;
        try { Thread.sleep(SLOW_MO_MS); } catch (InterruptedException ignored) {}
    }
}
```

```java
public class CreateUserTest {
    private WebElement waitVisible(By locator) {
        WebElement el = wait.until(ExpectedConditions.presenceOfElementLocated(locator));
        try {
            ((JavascriptExecutor) driver).executeScript( script: "arguments[0].scrollIntoView({block:'center'});", el);
        } catch (Exception ignored) {}
        return wait.until(ExpectedConditions.visibilityOf(el));
    }

    private boolean isPresent(By locator) {
        List<WebElement> els = driver.findElements(locator);
        return !els.isEmpty() && els.get(0).isDisplayed();
    }

    private void waitForLoaderToDisappear() {
        try {
            wait.until(ExpectedConditions.invisibilityOfElementLocated(loader));
        } catch (TimeoutException ignored) {
            // not fatal
        }
    }

    private void selectFromCustomDropdown(By dropdownDisplay, String optionText) {
        click(dropdownDisplay);
        By option = By.xpath( xpathExpression: "//div[@role='listbox']//span[normalize-space()='" + optionText + "']");
        click(option);
    }

    private void selectEmployeeFromAutocomplete(String partial) {
        clearAndType(employeeNameInput, partial);

        By firstOption = By.xpath( xpathExpression: "//div[@role='listbox']//span[1]");
        waitVisible(firstOption);
        click(firstOption);

        String value = driver.findElement(employeeNameInput).getAttribute( name: "value");
        Assert.assertTrue( condition: value != null && !value.trim().isEmpty(),
                message: "Employee Name was not selected properly from autocomplete.");
    }

    private By rowByUsername(String username) {
        return By.xpath( xpathExpression: "//div[@class='oxd-table-body']//div[@role='row']" +
                "[.//div[@role='cell'][2][normalize-space()='" + username + "']]");
```

```java
public class CreateUserTest {
    public void shouldCreateUserAndVerifyExistsInSearch() {
        // Wait for either our row or "No Records Found"
        By rowForUsername = rowByUsername(newUsername);
        wait.until( WebDriver d -> isPresent(rowForUsername) || isPresent(noRecordsFound));

        if (isPresent(noRecordsFound)) {
            Assert.fail("No Records Found after searching for username: " + newUsername);
        }

        // Assert exact match from the row that contains our username
        String foundUsername = driver.findElement(
                By.xpath( xpathExpression: "//div[@class='oxd-table-body']//div[@role='row']" +
                        "[.//div[@role='cell'][2][normalize-space()='" + newUsername + "']]" +
                        "//div[@role='cell'][2]")
        ).getText().trim();

        Assert.assertEquals(foundUsername, newUsername,
                message: "Expected the created username to appear in search results.");
    }

    // ---------- Robust Helpers ----------

    private void click(By locator) {  9 usages
        try {
            wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
        } catch (ElementClickInterceptedException | TimeoutException e) {
            WebElement el = wait.until(ExpectedConditions.presenceOfElementLocated(locator));
            ((JavascriptExecutor) driver).executeScript( script: "arguments[0].click();", el);
        }
        slowMo();
    }

    private void clearAndType(By locator, String text) {  7 usages
        WebElement el = waitVisible(locator);
        el.click();

        // Mac uses COMMAND+A; others use CONTROL+A
        Keys modifier = isMac() ? Keys.COMMAND : Keys.CONTROL;

        el.sendKeys(Keys.chord( ...value: modifier, "a"));
        el.sendKeys(Keys.BACK_SPACE);
        el.sendKeys(text);
        slowMo();
```

```java
    public class CreateUserTest {
        public void shouldCreateUserAndVerifyExistsInSearch() {
        // 2) Click Admin -> Add
        click(adminMenu);
        waitVisible(userManagementHeader);
        waitForLoaderToDisappear();

        click(addButton);
        waitVisible(addUserHeader);
        waitForLoaderToDisappear();

        // 3) Fill Add User form
        String newUsername = "user_" + UUID.randomUUID().toString().substring(0, 8);
        String password = "Test@" + UUID.randomUUID().toString().substring(0, 8) + "a1";

        selectFromCustomDropdown(userRoleDropdown,  optionText: "ESS");
        selectEmployeeFromAutocomplete( partial: "a");
        selectFromCustomDropdown(statusDropdown,  optionText: "Enabled");

        clearAndType(usernameInput, newUsername);
        clearAndType(passwordInput, password);
        clearAndType(confirmPasswordInput, password);

        // Save
        click(saveButton);

        // 4) Back to System Users page — wait until it is truly ready
        waitVisible(systemUsersHeader);
        waitForLoaderToDisappear();

        // The important part: wait until the username search input is clickable
        wait.until(ExpectedConditions.elementToBeClickable(searchUsernameInput));

        // Reset filters (optional but helps flakiness)
        if (isPresent(resetButton)) click(resetButton);

        // Search for our username
        clearAndType(searchUsernameInput, newUsername);
        click(searchButton);

        // Wait for either our row or "No Records Found"
        By rowForUsername = rowByUsername(newUsername);
        wait.until( WebDriver d -> isPresent(rowForUsername) || isPresent(noRecordsFound));
```

```java
public class CreateUserTest {
    // User management search (more robust: find the search form under System Users)
    private final By searchForm = By.xpath( xpathExpression: "//h5[normalize-space()='System Users']/ancestor::div[contains(@class,'oxd-table-filter')]");  no usages

    private final By searchUsernameInput = By.xpath(  2 usages
            xpathExpression: "//h5[normalize-space()='System Users']" +
                    "/ancestor::div[contains(@class,'orangehrm-background-container') or contains(@class,'orangehrm-paper-container')][1]" +
                    "//label[normalize-space()='Username']/../following-sibling::div/input"
    );

    private final By searchButton = By.xpath( xpathExpression: "//button[normalize-space()='Search']");  1 usage
    private final By resetButton = By.xpath( xpathExpression: "//button[normalize-space()='Reset']");  2 usages
    private final By noRecordsFound = By.xpath( xpathExpression: "//span[normalize-space()='No Records Found']");  2 usages

    // loader (may or may not appear)
    private final By loader = By.cssSelector(".oxd-loading-spinner");  1 usage

    @BeforeMethod
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        wait = new WebDriverWait(driver, Duration.ofSeconds(20));
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown() {
        if (driver != null) driver.quit();
    }

    @Test
    public void shouldCreateUserAndVerifyExistsInSearch() {
        driver.get(BASE_URL);

        // 1) Login
        clearAndType(By.name("username"), ADMIN_USER);
        clearAndType(By.name("password"), ADMIN_PASS);
        click(By.cssSelector("button[type='submit']"));

        // Wait for dashboard/homepage to load
        waitVisible(sidebar);
        waitForLoaderToDisappear();

        // 2) Click Admin -> Add
        click(adminMenu);
```

```java
package orangeHRM.tests;

import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.*;
import org.testng.Assert;
import org.testng.annotations.*;

import java.time.Duration;
import java.util.List;
import java.util.UUID;

public class CreateUserTest {

    private WebDriver driver;  11 usages
    private WebDriverWait wait;  8 usages

    // Toggle this if you want to visually slow down the run (for demo purposes)
    private static final boolean SLOW_MO = false;  1 usage
    private static final long SLOW_MO_MS = 250;  1 usage

    private static final String BASE_URL =  1 usage
            "https://opensource-demo.orangehrmlive.com/web/index.php/auth/login";
    private static final String ADMIN_USER = "Admin";  1 usage
    private static final String ADMIN_PASS = "admin123";  1 usage

    // ---------- Common Locators ----------
    private final By sidebar = By.cssSelector("aside");  1 usage
    private final By adminMenu = By.xpath( xpathExpression: "//span[normalize-space()='Admin']");  1 usage
    private final By userManagementHeader = By.xpath( xpathExpression: "//h6[normalize-space()='User Management']");  1 usage
    private final By addButton = By.xpath( xpathExpression: "//button[normalize-space()='Add']");  1 usage
    private final By addUserHeader = By.xpath( xpathExpression: "//h6[normalize-space()='Add User']");  1 usage
    private final By systemUsersHeader = By.xpath( xpathExpression: "//h5[normalize-space()='System Users']");  1 usage

    // Add User form fields
    private final By userRoleDropdown = By.xpath( xpathExpression: "//label[normalize-space()='User Role']/../following-sibling::div//div[contains(@class,'oxd-select-text')]");  1 usage
    private final By employeeNameInput = By.xpath( xpathExpression: "//label[normalize-space()='Employee Name']/../following-sibling::div//input");  2 usages
    private final By statusDropdown = By.xpath( xpathExpression: "//label[normalize-space()='Status']/../following-sibling::div//div[contains(@class,'oxd-select-text')]");  1 usage
    private final By usernameInput = By.xpath( xpathExpression: "//label[normalize-space()='Username']/../following-sibling::div/input");  1 usage
    private final By passwordInput = By.xpath( xpathExpression: "//label[normalize-space()='Password']/../following-sibling::div/input");  1 usage
    private final By confirmPasswordInput = By.xpath( xpathExpression: "//label[normalize-space()='Confirm Password']/../following-sibling::div/input");  1 usage
    private final By saveButton = By.xpath( xpathExpression: "//button[normalize-space()='Save']");  1 usage

    // User management search (more robust: find the search form under System Users)
```

testng.xml

```xml
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" >
<suite name="OrangeHRM Suite">
    <test name="Create User Test">
        <classes>
            <class name="orangeHRM.tests.CreateUserTest" />
        </classes>
    </test>
</suite>
```

pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>homework</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>8</maven.compiler.source>
        <maven.compiler.target>8</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.25.0</version>
        </dependency>

        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.11.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.5</version>
                <configuration>
                    <suiteXmlFiles>
                        <suiteXmlFile>testng.xml</suiteXmlFile>
                    </suiteXmlFiles>
                </configuration>
            </plugin>
        </plugins>
```